
KENSHI ONLINE

Entity Management Engine

Complete Technical Specification

The central nervous system of the multiplayer mod.
Every networked entity — players, NPCs, buildings, containers — flows through this system.

Document	Entity Management Engine — Technical Specification
Version	1.0
Date	February 2026
Classification	CONFIDENTIAL
Transport Layer	ENet (Reliable UDP)
Architecture	Listen-Server / Host-Authoritative
Target	Kenshi 1.x (32-bit, DLL Injection)

This document contains proprietary technical information.

Table of Contents

1 System Overview and Design Philosophy

2 Entity Data Model

- 2.1 NetworkedEntity Structure
- 2.2 Entity Types and ID Ranges
- 2.3 Entity State Machine
- 2.4 Dirty Flags and Delta Detection

3 Entity Registry

- 3.1 Registry Architecture
- 3.2 Thread Safety Model
- 3.3 ID Allocation Strategy
- 3.4 Entity Lookup and Queries

4 Entity Lifecycle

- 4.1 Local Entity Registration
- 4.2 Remote Entity Spawning (Ghost System)
- 4.3 Entity Despawning
- 4.4 Peer Disconnect Cleanup
- 4.5 Full Lifecycle Sequence Diagrams

5 Interpolation Engine

- 5.1 Snapshot Ring Buffer
- 5.2 Interpolation Algorithm
- 5.3 Extrapolation and Dead Reckoning
- 5.4 Snap Correction
- 5.5 Adaptive Delay (Jitter Estimation)
- 5.6 Teleport Detection

6 Authority Model

- 6.1 Authority Types
- 6.2 Authority Rules by Entity Type
- 6.3 Zone-Based NPC Authority
- 6.4 Zone Handoff Protocol
- 6.5 Hysteresis and Thrash Prevention

7 Game State Synchronization

- 7.1 Position and Movement
- 7.2 Animation State
- 7.3 Health and Limb System

- 7.4 Combat State
- 7.5 Inventory and Equipment
- 7.6 Squad System
- 7.7 Faction and World State
- 7.8 Buildings and Construction

8 Network Integration

- 8.1 Packet Structures
- 8.2 Channel Mapping
- 8.3 Send Rate and Dirty Coalescing
- 8.4 Receive Pipeline
- 8.5 World Snapshot on Join

9 Callback Architecture

- 9.1 Callback Interface
- 9.2 Hook Integration Points
- 9.3 Wiring to Offset Manager
- 9.4 Wiring to NPC Factory

10 The Main Tick Loop

- 10.1 Ghost Tick (Remote Entities)
- 10.2 Local Tick (Owned Entities)
- 10.3 Zone Authority Recalculation
- 10.4 Tick Ordering and Dependencies

11 Error Handling and Recovery

12 Performance Analysis

13 Memory Layout and Structure Sizes

14 API Reference

15 Implementation Checklist

1. System Overview and Design Philosophy

The Entity Management Engine is the **central nervous system** of the Kenshi Online multiplayer mod. It sits between the raw memory hooks (which read/write Kenshi's game entities) and the ENet network layer (which sends/receives packets). Every single networked object in the world — your player character, remote players, NPCs, buildings, containers — exists as a record inside this engine's registry.

Without this system, you're doing what most first attempts at multiplayer mods do: spawning player entities ad-hoc on connection, losing track of them, having no clear ownership model, and eventually hitting desync issues that are impossible to debug because there's no central source of truth about who owns what and what state each entity is in.

1.1 What This System Does

- **Assigns and manages NetworkIDs** — Every entity gets a unique 32-bit identifier that is consistent across all connected clients. The host allocates IDs. Clients use the IDs they receive.
- **Tracks entity lifecycle** — From spawn to despawn, every entity moves through a well-defined state machine: Inactive → Spawning → Active → Despawning → removed.
- **Creates ghost entities** — When a remote player connects, the engine spawns a local NPC puppet ('ghost') in your game world and drives it entirely from network data.
- **Runs interpolation** — Remote entities don't snap between network updates. A ring buffer of snapshots is maintained per entity, and the engine interpolates between them with adaptive delay based on observed jitter.
- **Manages authority** — Every entity has exactly one authoritative owner at any time. The owner reads state from game memory and sends it. Everyone else receives and writes it. Zone-based authority handles NPC ownership.
- **Detects dirty state** — Each tick, local entities are compared against their last-sent state. Only deltas are flagged for transmission, with rate limiting at 20Hz.
- **Provides thread-safe access** — The registry is accessed from hook threads (game thread), the network receive thread, and potentially UI threads. A `shared_mutex` provides concurrent read access with exclusive writes.

1.2 What This System Does NOT Do

- It does not manage the ENet connection itself (that's the Session Manager).
- It does not serialize packets (it calls a callback that your serialization layer implements).
- It does not install hooks or read raw memory (it calls a callback that your Offset Manager implements).
- It does not implement game-specific logic like damage formulas or AI (sync modules handle that).

The engine is deliberately **decoupled** from these systems via a callback interface. You wire it up by providing lambda functions that bridge to your existing hooks, offset manager, NPC factory, and ENet

wrapper. This means you can test the entity engine in isolation, swap out transport layers, or change how ghosts are spawned without modifying the core.

1.3 Design Principles

Principle	Implementation	Why It Matters
Single Source of Truth	One registry (<code>unordered_map</code>) holds ALL entity state.	Eliminates conflicting data. Any system can query the registry for the current authoritative state of any entity.
Authority-Based Updates	Each entity has an <code>AuthorityType</code> enum. Only the authority owner writes to game memory and sends updates.	Prevents two clients from simultaneously modifying the same NPC, which would cause desync.
Callback Decoupling	All external interactions go through an <code>EntityCallbacks</code> struct of <code>std::function</code> pointers.	The engine has zero dependencies on Kenshi internals, ENet, or any specific memory layout. Pure logic.
Rate-Limited Delta Sync	Dirty flags + 20Hz send timer = only changed data is sent, max 50ms between sends.	Minimizes bandwidth. A standing-still player sends nothing. Only movement/combat/state changes trigger packets.
Smooth Remote Rendering	<code>InterpBuffer</code> (ring buffer of 8 snapshots) + adaptive jitter-based delay + snap correction.	Remote players move smoothly even on lossy connections. No teleporting, no jitter, graceful degradation.
Thread Safety Without Bottlenecks	<code>shared_mutex</code> allows concurrent reads from multiple threads. Exclusive lock only for writes (spawn, despawn, authority change).	The game thread can read entity positions at 60Hz while the network thread pushes updates — no blocking.

2. Entity Data Model

Every networked entity in the world is represented by a single **NetworkedEntity** struct. This is the fundamental data type of the engine. It contains everything needed to identify, locate, replicate, interpolate, and manage an entity.

2.1 NetworkedEntity Structure

The struct is organized into logical groups:

Identity Block

Field	Type	Size	Description
netId	uint32_t	4B	Unique network identifier. Assigned by the host. 0 = invalid/broadcast.
type	EntityType (uint8_t)	1B	Player, NPC, Building, Container, or Squad.
state	EntityState (uint8_t)	1B	Current lifecycle state (Inactive, Spawning, Active, Despawning, Frozen).
displayName	std::string	~32B avg	Human-readable name shown in UI and debug logs.

Authority Block

Field	Type	Size	Description
authority	AuthorityType (uint8_t)	1B	None, Local, Remote, Host, or Transferring.
ownerPeerId	uint32_t	4B	The ENet peer ID of the authoritative owner.
currentZone	ZoneCoord (int32_t x2)	8B	Grid zone the entity currently occupies. Used for NPC authority partitioning.

Game Pointer Block

Field	Type	Size	Description
gameEntityPtr	void*	4B (32-bit)	Raw pointer to the Kenshi entity in memory. For local entities, this is the real player/NPC controller. For ghosts, it's the spawned puppet NPC.
isGhost	bool	1B	True if this entity is a remote replica driven by network data. False if it's a local entity we own.

Transform Block

Field	Type	Size	Description
position	Vec3 (3 floats)	12B	World-space position (x, y, z). Y is vertical in Kenshi.
rotation	Quat (4 floats)	16B	Orientation quaternion. Slerp'd during interpolation.
velocity	Vec3 (3 floats)	12B	Movement velocity vector. Used for extrapolation when packets are late.
moveSpeed	float	4B	Current movement speed scalar. Synced for animation blending.
animationId	uint16_t	2B	Current animation clip ID. Nearest-snapshot selection during interpolation.
animBlend	float	4B	Animation blend weight for transitions.

Game State Block

Field	Type	Size	Description
health	float	4B	Overall health value.
limbs	LimbHealth (7 floats)	28B	Per-limb HP: head, chest, stomach, leftArm, rightArm, leftLeg, rightLeg.
combat	CombatInfo	12B	Target NetID, attack state, block state, attack anim ID, attack timer.
factionId	uint32_t	4B	Which Kenshi faction this entity belongs to.
squadId	uint32_t	4B	Which squad this entity is assigned to.

Replication State Block

Field	Type	Size	Description
interpBuffer	InterpBuffer	~320B	Ring buffer of 8 EntitySnapshot records for interpolation.
jitter	JitterEstimator	~40B	Tracks packet arrival jitter for adaptive interpolation delay.
dirtyFlags	DirtyFlags (uint32_t)	4B	Bitfield of what has changed since last network send.
timeSinceLastSend	float	4B	Accumulator for rate-limiting outgoing updates (resets at 50ms).
lastRecvTick	uint16_t	2B	Server tick of last received packet. Used to reject out-of-order packets.
lastRecvTime	double	8B	Local timestamp of last received packet.
spawnTime	double	8B	Server time when this entity was registered/spawned.

Snap Correction Block

Field	Type	Size	Description
snapCorrectionStart	Vec3	12B	Position when a correction was triggered.

Field	Type	Size	Description
snapCorrectionTarget	Vec3	12B	Target position to correct towards.
snapCorrectionTimer	float	4B	Elapsed time during correction (0 to SNAP_CORRECT_MS).
snapCorrecting	bool	1B	True if a snap correction is currently in progress.

Total approximate size per entity: ~530 bytes. With 4096 max entities, the registry uses ~2.1 MB. This is negligible.

2.2 Entity Types and ID Ranges

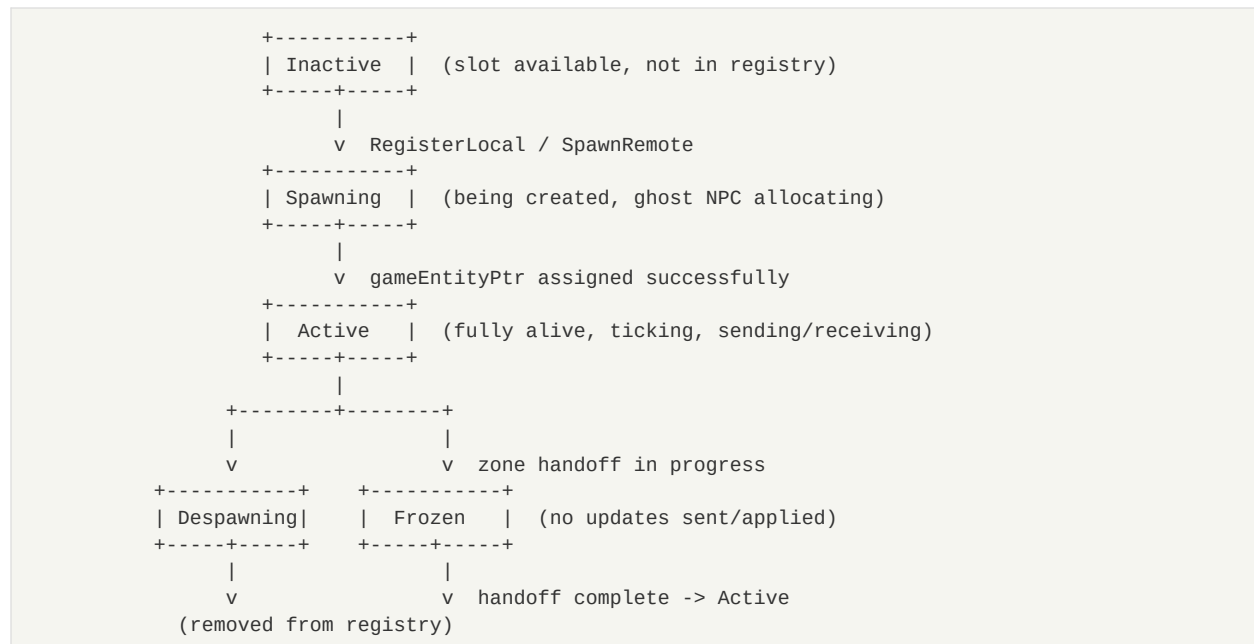
NetworkIDs are partitioned by entity type to allow fast type identification from just the ID:

Entity Type	Enum Value	NetworkID Range	Count	Purpose
Invalid	0	0	1	Reserved for broadcast/system packets.
Player	1	1 — 255	255	Human player characters. One per connected client.
NPC	2	256 — 8191	7936	All non-player characters: bandits, traders, animals, squad members.
Building	3	8192 — 16383	8192	Player-built structures, walls, gates, machines.
Container	4	16384 — 24575	8192	Loot bags, storage chests, shop inventories.
Squad	5	24576 — 32767	8192	Squad group entities (metadata, not physical).

The host maintains three monotonically increasing counters (`nextPlayerNetId`, `nextNPCNetId`, `nextBuildingNetId`) and allocates from the appropriate range. IDs are never reused within a session — if an entity is despawned, its ID is retired.

2.3 Entity State Machine

Every entity transitions through these states:



The **Frozen** state is used during zone authority handoffs. When a zone changes owner, all NPCs in that zone are frozen for up to 500ms while the old owner serializes their full state and the new owner receives and applies it. During this window, no position updates are sent or processed for those entities.

2.4 Dirty Flags and Delta Detection

The DirtyFlags bitfield tracks what has changed since the last network send:

Flag	Bit	Trigger Condition	Network Channel	Send Mode
Position	0x001	Position delta > 0.01 units	Channel 0 (unreliable)	20 Hz max
Rotation	0x002	Rotation delta > 0.5 degrees	Channel 0 (unreliable)	20 Hz max
Animation	0x004	animationId changed	Channel 0 (unreliable)	20 Hz max
Health	0x008	health value changed	Channel 1 (reliable)	On-change
Stats	0x010	Any stat (str, dex, etc.) changed	Channel 1 (reliable)	On-change
Inventory	0x020	Item added/removed	Channel 2 (reliable)	On-event
CombatState	0x040	Target, attack, or block state changed	Channel 1 (reliable)	On-change
LimbDamage	0x080	Any limb HP changed	Channel 1 (reliable)	On-change
SquadInfo	0x100	Squad membership or job changed	Channel 2 (reliable)	On-event
FactionRel	0x200	Faction relation changed	Channel 2 (reliable)	On-event
Equipment	0x400	Weapon/armor equipped/unequipped	Channel 2 (reliable)	On-event
AIState	0x800	NPC behavior state changed	Channel 1 (reliable)	On-change

Flags are **OR'd together** as changes are detected during TickLocal(). When timeSinceLastSend exceeds 50ms (20Hz) and dirtyFlags != None, the OnSendStateUpdate callback fires with the accumulated flags. After sending, ClearDirty() resets the bitfield to zero. This means multiple small changes within a 50ms window are coalesced into a single packet.

Critical: Dirty detection only runs for entities where authority == Local. Ghost entities (authority == Remote) never generate outgoing packets — they only consume incoming ones.

3. Entity Registry

3.1 Registry Architecture

The registry is a `std::unordered_map<uint32_t, NetworkedEntity>` stored as a member of the EntityManager singleton. The key is the NetworkID, the value is the full NetworkedEntity struct.

Design decisions and their rationale:

- **unordered_map over flat array:** With up to 16K possible IDs but typically only 50-200 active entities, a sparse map is far more memory-efficient. Hash lookups are $O(1)$ amortized.
- **Singleton pattern:** There is exactly one entity registry per game instance. The EntityManager::Get() static method returns a reference. This ensures all systems — hooks, network, UI — access the same data.
- **Reserve on init:** The map is reserve(4096) on initialization to avoid rehashing during gameplay.

3.2 Thread Safety Model

The EntityManager is accessed from at least three threads:

Thread	Access Pattern	Frequency	Operations
Game Thread (hooks)	Read + Write	60 Hz	TickLocal reads game memory, TickGhost writes game memory. Tick() is the main frame update.
Network Receive Thread	Write	20-60 Hz	OnPositionUpdate, OnHealthUpdate, etc. push data into ghost entities' interp buffers.
Network Send Thread	Read	20 Hz	OnSendStateUpdate callback reads entity state to serialize into packets.

Thread safety is provided by a `std::shared_mutex` (`m_mutex`). This allows:

- **Concurrent reads:** Multiple threads can hold a shared_lock simultaneously. The game thread reading positions while the send thread serializes state — no blocking.
- **Exclusive writes:** A unique_lock is required for structural modifications: spawning, despawning, authority changes. These block all readers temporarily.
- **Granularity choice:** We use a single mutex for the entire registry rather than per-entity locks. With <200 entities and <1ms per tick, contention is negligible. Per-entity locks would add 200+ mutex objects and complicate the code without measurable benefit.

Lock ordering rule: Always acquire `m_mutex` before any game memory access. Never hold `m_mutex` while calling into ENet (which has its own internal locks). The callback pattern naturally enforces this — callbacks fire with the lock held, but they should not block on external locks.

3.3 ID Allocation Strategy

Only the host allocates NetworkIDs. Clients receive their assigned ID during the handshake (HANDSHAKE_ACK packet). When the host needs to register a new entity:

1. Check the appropriate counter (`m_nextPlayerNetId`, `m_nextNPCNetId`, or `m_nextBuildingNetId`).
2. If the counter hasn't exceeded the range maximum, return the current value and increment.
3. If the range is exhausted, return `NETID_INVALID (0)`. The caller must handle this gracefully.

IDs are **never reused** within a session. If entity 300 (an NPC) is despawned, ID 300 is retired. The next NPC gets 301. This prevents stale references — if a packet arrives for ID 300 after it was despawned, the registry lookup simply returns `nullptr`, and the packet is silently dropped.

3.4 Entity Lookup and Queries

The EntityManager provides several query methods:

Method	Return Type	Lock Type	Description
<code>GetEntity(netId)</code>	<code>NetworkedEntity*</code>	None (caller holds)	Direct pointer lookup. $O(1)$. Returns <code>nullptr</code> if not found.
<code>GetLocalPlayer()</code>	<code>NetworkedEntity*</code>	None	Shortcut for <code>GetEntity(m_localPlayerNetId)</code> .
<code>GetEntitiesByType(type)</code>	<code>vector</code>	<code>shared_lock</code>	Returns all active entity IDs of the given type. $O(n)$.
<code>GetEntitiesInRadius(pos, r)</code>	<code>vector</code>	<code>shared_lock</code>	Spatial query. Returns all active entities within radius. $O(n)$.
<code>GetEntitiesInZone(zone)</code>	<code>vector</code>	<code>shared_lock</code>	Returns all entities in a specific authority zone. $O(n)$.
<code>ForEachEntity(callback)</code>	<code>void</code>	<code>shared_lock</code>	Iterates all entities with a const callback. Thread-safe.
<code>GetEntityCount()</code>	<code>size_t</code>	<code>shared_lock</code>	Total entity count.
<code>GetGhostCount()</code>	<code>size_t</code>	<code>shared_lock</code>	Count of active ghost (remote) entities.

Performance note: `GetEntitiesInRadius` is $O(n)$ over all entities. For v1.0 with <200 entities, this is fine (<0.01ms). If entity counts grow significantly in future versions, a spatial hash grid should be added.

4. Entity Lifecycle

This section covers every path an entity can take from creation to destruction. Understanding the lifecycle is critical because **every bug in multiplayer mods traces back to a lifecycle violation** — spawning an entity twice, despawning before all references are cleared, or updating an entity that hasn't finished spawning.

4.1 Local Entity Registration

When YOU (the local client) need to register an entity you own:

4.1.1 RegisterLocalPlayer

Called once when the player controller hook first fires:

1. Acquire **unique_lock** on m_mutex.
2. Determine NetworkID: if host, allocate from player range. If client, use the ID assigned during handshake.
3. Create a NetworkedEntity with type=Player, authority=Local, isGhost=false.
4. Call OnReadTransform callback to read the initial position from game memory.
5. Calculate the initial ZoneCoord from the position.
6. Insert into the registry: m_entities[netId] = entity.
7. Store the netId as m_localPlayerNetId for fast lookup.
8. Call OnBroadcastSpawn callback to notify all remote clients via ENTITY_SPAWN packet.
9. Return the assigned NetworkID.

Critical: RegisterLocalPlayer must be called exactly once. Calling it again will overwrite the existing entity. The hook should check if GetLocalPlayerNetId() != NETID_INVALID before calling.

4.1.2 RegisterLocalNPC

Same flow as RegisterLocalPlayer but for squad members or world NPCs you own:

- Uses NPC ID range instead of player range.
- Accepts an optional squadId parameter.
- Multiple NPCs can be registered (your full squad).
- Each gets a unique NetworkID and is broadcast to all clients.

4.2 Remote Entity Spawning (Ghost System)

When a remote client's entity needs to appear in your world, the engine creates a **ghost** — a real NPC entity in Kenshi's game world, but with AI disabled and movement driven entirely by network packets.

4.2.1 SpawnRemoteEntity Full Flow

1. Acquire **unique_lock**.
2. Check if the NetworkID already exists. If so and it's Active, return true (idempotent). If it's Despawning, re-activate it.
3. Create NetworkedEntity with state=Spawning, authority=Remote, isGhost=true.
4. Call **OnSpawnGhost** callback. This is where your NPC factory creates a real Kenshi NPC entity using the controller stubs.
5. The callback returns a void* pointer to the newly created game entity. If nullptr, spawn failed — return false.
6. Store the gameEntityPtr. Push an initial EntitySnapshot into the interpolation buffer with the spawn position.
7. Reset the JitterEstimator.
8. Transition state to Active.
9. Insert into registry.

4.2.2 Ghost NPC Creation (Inside OnSpawnGhost Callback)

The OnSpawnGhost callback must:

- Call Kenshi's internal NPC allocation function via the known offset/stub.
- Set the NPC's model and appearance to match the remote player's data.
- **Disable AI completely** — zero out the AI state pointer or redirect it to a no-op behavior tree. If AI is left active, the ghost NPC will try to pathfind and fight independently, causing visual chaos.
- Position the NPC at the spawn location.
- Return the game entity pointer.

Critical: The ghost NPC must have AI disabled BEFORE it's registered as Active. If there's even one frame where the AI runs, it may move the ghost to a different position, which will conflict with the first network position update and cause a visible snap.

4.3 Entity Despawning

DespawnEntity(netId) performs:

1. Acquire unique_lock.
2. Find entity in registry. If not found, return (no-op).
3. Set state to Despawning.
4. If it's a ghost and gameEntityPtr is valid, call OnDespawnGhost to remove the NPC from the game world.
5. If we're the authority owner, call OnBroadcastDespawn to notify all clients.

6. Erase from the registry. The NetworkedEntity is destroyed.

4.4 Peer Disconnect Cleanup

When ENet detects a peer disconnect (graceful or timeout), **DespawnPeerEntities(peerId)** is called. This iterates the entire registry, collects all ghost entities owned by that peer, and despawns each one. This ensures no orphaned ghosts remain in the world.

The method unlocks the mutex before calling DespawnEntity for each entity (since DespawnEntity acquires its own lock). This prevents a deadlock but means the iteration and despawn are not atomic — which is acceptable since no new entities can be spawned for a disconnected peer.

4.5 Full Lifecycle Sequence Diagrams

4.5.1 Client Joins (from Host's perspective)

```
ENet: New peer connected (peerId=3)
|
v
Host receives HANDSHAKE_REQ (version, name, mod hash)
|
|-- Validate version and mod hash
|-- Allocate NetworkID (e.g., netId=3) from player range
|-- Send HANDSHAKE_ACK to peer 3 (assigned netId=3, server time)
|
v
Host calls GetWorldSnapshot()
|
|-- Serialize all active entities
|-- Send ENTITY_SPAWN packet for each to peer 3
|
v
Host waits for peer 3's first POS_UPDATE
|
|-- OnPositionUpdate triggers SpawnRemoteEntity on host
|-- Ghost for peer 3's player appears in host's world
|-- Host broadcasts ENTITY_SPAWN for the new player to all OTHER peers
```

4.5.2 Entity During Normal Gameplay

```
Each frame (60 Hz):
|
|-- EntityManager::Tick(deltaTime) called from game thread
|
|-- For each LOCAL entity:
|   |-- OnReadTransform(gameEntityPtr) -> read position from game memory
|   |-- Compare with stored state -> set DirtyFlags
|   |-- If dirty and 50ms elapsed -> OnSendStateUpdate callback
|   |-- ClearDirty, reset timer
|
|-- For each GHOST entity:
|   |-- Calculate renderTime = serverTime - adaptiveDelay
|   |-- Sample interpolation buffer at renderTime
|   |-- Apply snap correction if active
|   |-- OnWriteTransform(gameEntityPtr, interpPos, interpRot, ...)
|
|-- Every 1 second (host only):
|   |-- RecalculateZoneAuthority()
|   |-- Fire OnAuthorityChanged for any zone transfers
```

5. Interpolation Engine

The interpolation engine is what makes remote players look **smooth** instead of teleporting between 20Hz network updates while you render at 60Hz. It's the difference between a professional multiplayer experience and a slideshow.

5.1 Snapshot Ring Buffer

Each ghost entity has an InterpBuffer — a fixed-size ring buffer of EntitySnapshot records:

Property	Value	Rationale
Buffer Size	8 snapshots	At 20Hz, 8 snapshots = 400ms of history. Enough to survive short packet bursts and jitter spikes.
Write Index	Monotonically increasing	Wraps via modulo. Oldest snapshot is silently overwritten.
Snapshot Contents	timestamp, tick, pos, rot, vel, animId, animBlend, moveSpeed	Everything needed to fully reconstruct the entity's visual state at a point in time.
Sort Behavior	Sorted by timestamp on read	Snapshots may arrive slightly out of order due to network jitter. The Sample() method sorts before interpolating.

5.2 Interpolation Algorithm

The core interpolation algorithm runs every frame in TickGhost():

1. **Calculate render time:** $\text{renderTime} = \text{serverTime} - \text{adaptiveDelay}$. The adaptive delay is provided by the JitterEstimator (see 5.5).
2. **Gather and sort:** Collect all valid snapshots from the ring buffer and sort by timestamp.
3. **Find bracket:** Find two consecutive snapshots where $\text{snapshot}[i].\text{timestamp} \leq \text{renderTime} \leq \text{snapshot}[i+1].\text{timestamp}$.
4. **Calculate t:** $t = (\text{renderTime} - \text{snapshot}[i].\text{timestamp}) / (\text{snapshot}[i+1].\text{timestamp} - \text{snapshot}[i].\text{timestamp})$. Clamped to [0, 1].
5. **Interpolate position:** $\text{Vec3}::\text{Lerp}(\text{snap}[i].\text{position}, \text{snap}[i+1].\text{position}, t)$.
6. **Interpolate rotation:** $\text{Quat}::\text{Slerp}(\text{snap}[i].\text{rotation}, \text{snap}[i+1].\text{rotation}, t)$. Handles quaternion sign and near-identity cases.
7. **Select animation:** Use $\text{snap}[i].\text{animationId}$ if $t < 0.5$, else $\text{snap}[i+1].\text{animationId}$. No blending between different animation clips.
8. **Interpolate scalars:** Linear interpolation for animBlend, moveSpeed, velocity.

***Why Slerp for rotation?** Linear interpolation of quaternions (nlerp) can produce non-unit quaternions and variable angular velocity. Slerp maintains constant angular velocity and unit length. For small rotations (the common case in Kenshi), the performance difference is negligible.*

5.3 Extrapolation and Dead Reckoning

When the render time exceeds all available snapshots (packets are late), the engine extrapolates using the last known velocity vector:

- Extrapolated position = $\text{lastSnapshot.position} + \text{lastSnapshot.velocity} * dt$
- dt is clamped to **250ms maximum** (EXTRAP_MAX_MS). Beyond 250ms, the entity freezes in place.
- Rotation is held constant during extrapolation (predicting rotation changes is unreliable).
- Animation continues playing the last known clip.

The 250ms cap prevents runaway drift. If a player was running north when a packet was lost, without the cap they'd extrapolate north forever. With the cap, they slide 250ms of distance (~2.5 meters at sprint speed) then freeze. When the next real packet arrives, snap correction smoothly pulls them to the correct position.

5.4 Snap Correction

When a new packet arrives after extrapolation (or after a mid-range position discrepancy), the entity's current visual position may differ from the new authoritative position. Instead of teleporting:

1. Detect the distance between the current visual position and the new target.
2. If distance > 50 units: **teleport** immediately (the entity was probably loaded into a new area).
3. If distance > 5 units but <= 50: activate **snap correction**. Store the current position as snapCorrectionStart and the target as snapCorrectionTarget.
4. Over the next **150ms** (SNAP_CORRECT_MS), lerp from start to the ongoing interpolation target.
5. If distance <= 5 units: normal interpolation handles it naturally. No correction needed.

5.5 Adaptive Delay (Jitter Estimation)

The display delay (how far behind 'real time' we render ghost entities) adapts per-entity based on observed network jitter. The JitterEstimator tracks the difference between expected and actual packet arrival intervals using an exponential moving average:

```
On each packet arrival:
  expectedDelta = serverTimestamp - lastServerTimestamp
  actualDelta   = localReceiveTime - lastLocalReceiveTime
  instantJitter = |actualDelta - expectedDelta|
  avgJitter     = avgJitter * 0.9 + instantJitter * 0.1

Delay mapping:
  avgJitter < 20ms -> delay = 50ms   (tight, responsive)
  avgJitter > 80ms -> delay = 200ms  (buffered, smooth)
  in between       -> linear scale between 50ms and 200ms
```

The delay is recalculated on every packet arrival (not on a timer). This means it responds quickly to changing network conditions. A player on a stable LAN connection gets 50ms delay (very responsive). A player on a lossy Wi-Fi connection gets 150-200ms delay (slightly laggy but smooth).

5.6 Teleport Detection

Kenshi has loading zones and AI teleportation. The engine must distinguish between smooth movement and teleportation:

Distance Jump	Action	Rationale
<= 5 units	Normal interpolation	Normal movement at any speed.
5 — 50 units	Snap correction (150ms blend)	Accumulated error or brief packet loss. Smooth correction.
> 50 units	Instant teleport	Loading zone, fast travel, or AI teleport. Interpolating would look absurd.

6. Authority Model

Authority answers the question: **who is the source of truth for this entity's state?** In a multiplayer game, if two clients both try to update the same NPC, you get conflicting states and desync. The authority model prevents this by assigning exactly one owner to each entity.

6.1 Authority Types

Type	Value	Meaning	Tick Behavior
None	0	No authority. Entity exists but isn't being updated by anyone.	Skip.
Local	1	WE own this entity. We read from game memory and send updates.	TickLocal: read state, detect dirty, send.
Remote	2	Another peer owns this entity. We receive updates and display them.	TickGhost: interpolate, write to game memory.
Host	3	The host owns this entity exclusively (buildings, world state).	Same as Local on host; same as Remote on clients.
Transferring	4	Authority is being transferred (zone handoff in progress).	Frozen — no reads or writes until handoff completes.

6.2 Authority Rules by Entity Type

Entity Type	Default Authority	Can Transfer?	Transfer Trigger
Local Player Character	Local (always the owning client)	Never	N/A — player always owns their own character.
Player's Squad Members	Local (the player who recruited them)	Yes, on trade/dismiss	When an NPC is traded to another player or dismissed.
World NPCs (not in any player squad)	Zone owner	Yes, on zone handoff	When a closer player enters the zone.
Buildings	Host (always)	Never	Host is always authoritative for all buildings.
Containers / Loot	Host (arbitrates locks)	Never	Lock-based access. Host decides who can modify.
World State / Factions	Host (always)	Never	Only the host modifies faction relations, town ownership, etc.

6.3 Zone-Based NPC Authority

The world map is divided into a grid of **512 x 512 game unit zones**. Each zone is assigned to the player whose character is closest to the zone's center. That player simulates all world NPCs (non-squad) in that

zone — running their AI, pathfinding, combat decisions — and broadcasts the resulting state to all other clients.

Why zone-based? Because Kenshi's world is enormous (~870 km²) but NPCs are local. A bandit patrol in the desert doesn't affect a player in the swamp. Zone authority means each player only simulates NPCs near them, distributing CPU load.

Zone coordinates are calculated as: zoneX = floor(worldX / 512), zoneZ = floor(worldZ / 512). Each zone is represented by a ZoneCoord struct (two int32 values).

6.4 Zone Handoff Protocol

When the host detects that a new player is now closest to a zone:

```
Host detects: Player B is now closer to zone (3, 7) than Player A
|
|-- Check hysteresis: is B closer by > 64 units? If not, skip.
|
v (hysteresis passed)
Host sets all NPCs in zone (3,7) to state=Frozen
|
|-- Sends ZONE_HANDOFF_BEGIN to Player A (old owner) and Player B (new owner)
|
v
Player A serializes full state of all NPCs in zone (3,7):
- Position, rotation, velocity
- Health, limb HP
- AI state, current behavior, patrol waypoint
- Inventory contents
- Combat target and state
|
|-- Sends NPC_STATE_DUMP to Host
|-- Host relays to Player B
|
v
Player B receives NPC_STATE_DUMP:
- Creates or updates local NPC entities with the received state
- Switches them to authority=Local
- Begins AI simulation for those NPCs
|
|-- Sends ZONE_HANDOFF_ACK to Host
|
v
Host broadcasts ZONE_HANDOFF_COMPLETE:
- All clients now accept updates from Player B for zone (3,7) NPCs
- NPCs transition from Frozen back to Active
- Total handoff time target: < 500ms
```

6.5 Hysteresis and Thrash Prevention

Without hysteresis, two players standing near a zone boundary would constantly trade authority back and forth as they shift position slightly. This 'thrashing' causes NPCs to freeze and unfreeze rapidly.

The solution: a new player must be closer by at least **64 game units** (AUTHORITY_HYSTERESIS) before a transfer triggers. This creates a 'dead zone' where the current owner retains authority even if they're no

longer the absolute closest player. Only a significant position change causes a transfer.

7. Game State Synchronization

This section details how each specific Kenshi game mechanic is synchronized across clients.

7.1 Position and Movement

Property	Value
Channel	0 (Unreliable Sequenced)
Packet	POS_UPDATE (0x10)
Send Rate	20 Hz (every 50ms)
Payload Size	44 bytes (3 pos + 4 rot + 3 vel + animId + animBlend + moveSpeed)
Bandwidth Per Entity	~880 B/s = 7 kbps
Dirty Threshold	Position delta > 0.01 units OR rotation delta > 0.5 degrees
Receive Handling	Push into InterpBuffer. Reject out-of-order via tick comparison.

Position updates are the highest-frequency and lowest-latency data in the system. Using unreliable-sequenced delivery means late packets are silently dropped (ENet handles this) and the interpolation buffer naturally handles gaps.

7.2 Animation State

Animation is sent alongside position in the POS_UPDATE packet:

- **animationId** (uint16_t): The current animation clip index. Mapped to Kenshi's internal animation enum.
- **animBlend** (float): Blend weight for transitions between clips (0.0 = previous clip, 1.0 = current clip).
- During interpolation, the animationId is selected by nearest-snapshot (not interpolated). Blend weight is interpolated.
- Attack animations are handled specially via COMBAT_STATE on the reliable channel (see 7.4).

7.3 Health and Limb System

Kenshi tracks health at two levels: overall HP and per-limb HP for 7 body parts. The limb system is **the most gameplay-critical sync** because limbs reaching 0 causes severing, KO, or death.

Limb ID	Body Part	Gameplay Effect at 0 HP
0	Head	Instant death.
1	Chest	Knockout. Can be picked up by allies or enemies.
2	Stomach	Knockout. Hunger drain massively increased.

Limb ID	Body Part	Gameplay Effect at 0 HP
3	Left Arm	Severed. Cannot dual-wield. Reduced combat ability.
4	Right Arm	Severed. Cannot use two-handed weapons effectively.
5	Left Leg	Severed. Movement speed drastically reduced. Requires prosthetic.
6	Right Leg	Severed. Same as left leg.

Limb damage is sent as LIMB_DAMAGE (0x22) packets on the reliable channel. Only the host sends these — the host calculates all damage via the combat system and distributes the results.

7.4 Combat State

Combat is **host-authoritative**. Clients send attack intent; the host resolves hits and damage.

The CombatInfo struct synced per entity contains:

- **targetNetId**: The NetworkID of the entity being attacked. 0 = no target.
- **attackState**: 0=idle, 1=windup, 2=swing, 3=recover.
- **blockState**: 0=none, 1=blocking, 2=staggered.
- **attackAnimId**: Which attack animation is playing.

Combat flow:

1. Attacker's client detects attack via hook. Sends COMBAT_STATE (reliable, channel 1).
2. Host receives. Validates range, line of sight, weapon type vs target armor.
3. Host applies Kenshi's damage formula. Sends DAMAGE_EVENT (reliable, channel 2) to all clients.
4. Host sends LIMB_DAMAGE for the specific limb hit.
5. All clients play hit reaction animation on the target ghost.
6. If the attacker's client had optimistically played the attack hit, and the host says 'miss' or 'blocked', the client receives no DAMAGE_EVENT and plays a correction (block animation on target).

7.5 Inventory and Equipment

Inventory sync uses a lock-based protocol for shared containers:

- **Player inventory**: Synced on-change. The authoritative client sends item add/remove events.
- **Containers (chests, shops)**: Lock-based. A player sends CONTAINER_LOCK request to the host. Host grants/denies. While locked, only the locking player can modify the container.
- Lock timeout: **30 seconds** or when the player closes the inventory UI, whichever comes first.
- Equipment changes (weapon swap, armor equip) are sent as reliable events and trigger visual updates on ghost entities.

7.6 Squad System

- **SQUAD_UPDATE** packets contain: squadId, list of member NetworkIDs, job assignments per member, formation flags.
- Sent when membership changes (recruit, dismiss, death) or job assignments change.
- Each player manages their own squad independently in v1.0.

7.7 Faction and World State

- Host-authoritative. All faction relation changes, town ownership, bounties, and world event flags are managed by the host.
- FACTION_UPDATE packets broadcast changes to all clients.
- Full faction state is included in the WORLD_SNAPSHOT sent to new clients on join.

7.8 Buildings and Construction

- Host-authoritative. Players send build requests; host validates and broadcasts.
- Construction progress is tracked as a STAT_DELTA on the building entity.
- Building damage from attacks is calculated by the host and broadcast.

8. Network Integration

8.1 Packet Structures

All packets share a common 8-byte header:

```
PacketHeader (8 bytes, packed):
uint8_t  type;           // Packet type enum
uint8_t  flags;          // 0x01=compressed, 0x02=fragmented
uint16_t tick;           // Server tick counter (wraps at 65535)
uint32_t netId;          // Target entity NetworkID (0=broadcast)
```

Key payload structures:

```
PosUpdatePayload (44 bytes, packed):
float posX, posY, posZ;    // 12B World position
float rotX, rotY, rotZ, rotW; // 16B Orientation quaternion
float velX, velY, velZ;    // 12B Velocity for extrapolation
uint16_t animId;           // 2B Current animation clip
float animBlend;           // 4B Animation blend weight
float moveSpeed;           // 4B Movement speed scalar

EntitySpawnPayload (108 bytes, packed):
uint32_t netId;            // 4B Assigned NetworkID
uint8_t  entityType;       // 1B EntityType enum
uint32_t ownerPeerId;      // 4B Authoritative peer
float    posX, posY, posZ;  // 12B Spawn position
float    rotX, rotY, rotZ, rotW; // 16B Spawn rotation
float    health;            // 4B Initial health
char     displayName[32];   // 32B Entity name (null-terminated)
char     modelId[32];       // 32B Model identifier

DamageEventPayload (13 bytes, packed):
uint32_t attackerNetId;    // 4B Who dealt the damage
uint32_t targetNetId;      // 4B Who received the damage
float    damage;           // 4B Damage amount
uint8_t  limbId;           // 1B Which limb was hit (0-6)
```

8.2 Channel Mapping

Channel	ENet Mode	Purpose	Typical Packets
0	Unreliable Sequenced	High-frequency spatial data	POS_UPDATE, ANIM_EVENT
1	Reliable Ordered	Medium-frequency state changes	STAT_DELTA, COMBAT_STATE, LIMB_DAMAGE, HEALTH_UPDATE
2	Reliable Ordered	Low-frequency events	DAMAGE_EVENT, DEATH_EVENT, ITEM_PICKUP, BUILDING_PLACE, SQUAD_UPDATE, FACTION_UPDATE, CHAT_MSG

Channel	ENet Mode	Purpose	Typical Packets
3	Reliable Ordered	Control / lifecycle	HANDSHAKE_REQ/ACK, WORLD_SNAPSHOT, ENTITY_SPAWN, ENTITY_DESPAWN, DISCONNECT, ZONE_HANDOFF

8.3 Send Rate and Dirty Coalescing

The send system is rate-limited to **20 Hz** (50ms intervals) per entity. Here's why this matters:

- Kenshi runs at 30-60 FPS. Reading entity state every frame would mean 30-60 potential updates per second per entity.
- Most of those updates are redundant (position changed by < 0.001 units between frames).
- By coalescing dirty flags over 50ms windows, we send one packet that captures all changes, instead of 3-4 tiny packets.
- For 8 players each with 5 squad NPCs = 48 entities. At 20Hz, that's 960 packets/second. At 60Hz, it would be 2880 — triple the overhead for no visual improvement.

8.4 Receive Pipeline

Incoming packets flow through:

1. ENet poll loop dequeues received packets.
2. PacketHeader is parsed. Type byte routes to the correct handler.
3. Payload is memcpy'd into the appropriate payload struct (packed, no alignment issues).
4. The handler calls the appropriate EntityManager method (OnPositionUpdate, OnHealthUpdate, etc.).
5. The EntityManager acquires a shared_lock and updates the target entity.

8.5 World Snapshot on Join

When a new client connects, the host calls **GetWorldSnapshot()** which iterates all active entities and returns a vector of EntitySpawnData structs. The host then serializes each as an ENTITY_SPAWN packet and sends them all to the new client on channel 3 (reliable, ordered). This ensures the new client has a complete picture of the world before they start receiving incremental updates.

9. Callback Architecture

The `EntityManager` communicates with the outside world exclusively through the **EntityCallbacks** struct. This is a collection of `std::function` pointers that you assign during initialization. The engine calls them at the appropriate times. You implement them by bridging to your existing systems.

9.1 Callback Interface

Callback	Signature	When It Fires	What You Implement
<code>OnSpawnGhost</code>	<code>void*(netId, type, pos, modelId)</code>	When a remote entity needs a visual puppet	Call NPC factory stubs. Create a Kenshi NPC. Disable AI. Return <code>gameEntityPtr</code> .
<code>OnDespawnGhost</code>	<code>void(gameEntityPtr)</code>	When a ghost entity is being removed	Call NPC destructor. Free the Kenshi NPC.
<code>OnWriteTransform</code>	<code>void(ptr, pos, rot, animId, blend, speed)</code>	Every frame per ghost entity	Write interpolated values to game memory via Offset Manager.
<code>OnReadTransform</code>	<code>EntitySnapshot(gameEntityPtr)</code>	Every frame per local entity	Read current state from game memory via Offset Manager.
<code>OnAuthorityChanged</code>	<code>void(netId, oldOwner, newOwner)</code>	When zone authority transfers	Send <code>ZONE_HANDOFF</code> packets. Update local entity authority.
<code>OnSendStateUpdate</code>	<code>void(netId, flags, entity)</code>	When a local entity has dirty state to send	Serialize into packets. Send via ENet on appropriate channel.
<code>OnBroadcastSpawn</code>	<code>void(entity)</code>	When a local entity is registered	Serialize <code>ENTITY_SPAWN</code> . Broadcast to all peers.
<code>OnBroadcastDespawn</code>	<code>void(netId)</code>	When a local entity is removed	Serialize <code>ENTITY_DESPAWN</code> . Broadcast to all peers.

9.2 Hook Integration Points

Your existing trampoline hooks connect to the `EntityManager` at these points:

- **PlayerController::Update hook:** On first fire, call `RegisterLocalPlayer(controllerPtr)`. On subsequent fires, the `Tick()` loop handles reading state via `OnReadTransform`.
- **Game main loop hook:** Call `EntityManager::Tick(deltaTime)` once per frame. This drives the entire system — ghost interpolation, local dirty detection, zone authority, everything.
- **ENet receive callback:** Route packets to `OnPacketReceived()` which dispatches to the appropriate `EntityManager` method based on packet type.
- **ENet disconnect callback:** Call `DespawnPeerEntities(peerId)` to clean up all ghost entities for the disconnected player.

9.3 Wiring to Offset Manager

The OnReadTransform and OnWriteTransform callbacks bridge to your Offset Manager:

```
// OnReadTransform implementation
EntitySnapshot snap;
snap.position.x = OffsetManager::ReadFloat(ptr, "posX");
snap.position.y = OffsetManager::ReadFloat(ptr, "posY");
snap.position.z = OffsetManager::ReadFloat(ptr, "posZ");
snap.rotation.x = OffsetManager::ReadFloat(ptr, "rotX");
// ... etc for all transform fields
return snap;

// OnWriteTransform implementation
OffsetManager::WriteFloat(ptr, "posX", pos.x);
OffsetManager::WriteFloat(ptr, "posY", pos.y);
// ... etc. The interpolated values are written directly to game memory.
```

9.4 Wiring to NPC Factory

The OnSpawnGhost callback uses your NPC creation stubs:

```
// OnSpawnGhost implementation
void* npc = NPCFactory::SpawnPuppetNPC(pos, modelId);
if (!npc) return nullptr; // spawn failed
NPCFactory::DisableAI(npc); // CRITICAL: must disable before returning
return npc;
```

10. The Main Tick Loop

EntityManager::Tick(float deltaTime) is the heartbeat of the system. It's called once per frame from the game thread. Everything flows through here.

10.1 Ghost Tick (Remote Entities)

For each entity where `isGhost == true` and `state == Active`:

1. Get adaptive interpolation delay from the entity's `JitterEstimator`.
2. Calculate `renderTime = serverTime - delay`.
3. Call `interpBuffer.Sample(renderTime)` to get interpolated snapshot.
4. Check for snap correction: is there a large jump from current to interpolated position?
5. If snap correcting, lerp between correction start and interpolated target.
6. If teleport detected (> 50 units), snap immediately.
7. Call `OnWriteTransform` to push the final position/rotation/animation into game memory.

10.2 Local Tick (Owned Entities)

For each entity where `isGhost == false` and `state == Active`:

1. Call `OnReadTransform` to read current state from game memory.
2. Compare position to stored position. If $\text{delta} > 0.01$, set Position dirty flag.
3. Compare `animationId`. If changed, set Animation dirty flag.
4. Update stored state (position, rotation, velocity, animation).
5. Update `currentZone` from new position.
6. Accumulate `timeSinceLastSend += deltaTime`.
7. If `dirtyFlags != None` AND `timeSinceLastSend  $\geq$  50ms`: call `OnSendStateUpdate`, clear dirty, reset timer.

10.3 Zone Authority Recalculation

Host only. Runs every 1 second (not every frame — too expensive):

1. Collect all player positions and peer IDs.
2. For each zone that contains NPCs, find the closest player.
3. Check hysteresis: is the new closest player closer by > 64 units than the current owner?
4. If yes: update zone authority map. Set NPCs in zone to new owner. Fire `OnAuthorityChanged`.
5. If no: keep current owner (prevents thrashing).

10.4 Tick Ordering and Dependencies

Order	Operation	Thread	Dependency
1	ENet poll (receive packets)	Network Thread	None — runs independently
2	Incoming packets routed to EntityManager	Network Thread	Packets arrive before Tick
3	EntityManager::Tick(dt) — ghost interpolation	Game Thread	Interp buffers populated by step 2
4	EntityManager::Tick(dt) — local state read	Game Thread	Game memory updated by engine this frame
5	EntityManager::Tick(dt) — dirty send	Game Thread	State read in step 4
6	ENet flush (send queued packets)	Network Thread	Packets queued in step 5
7	Zone authority recalc (1/sec, host)	Game Thread	Player positions updated in step 4

11. Error Handling and Recovery

Error Scenario	Detection	Recovery Action	Data Impact
Ghost spawn fails (NPCFactory returns nullptr)	OnSpawnGhost returns nullptr	SpawnRemoteEntity returns false. Entity not added to registry. Remote peer's updates are silently dropped until next ENTITY_SPAWN.	No local entity exists. Remote player is invisible. Recoverable on next spawn attempt.
Out-of-order position packet	serverTick < lastRecvTick (int16 comparison)	Packet is silently dropped. The interpolation buffer is unaffected.	None. Old data never overwrites new data.
Stale entity update (entity already despawned)	Registry lookup returns nullptr	Update is silently dropped.	None.
NetworkID range exhausted	Allocator returns NETID_INVALID	Entity cannot be registered. Log error. For players, reject connection. For NPCs, skip sync.	Entity exists in game but not in network. Only affects local client.
Host disconnects	All clients detect ENet timeout	All ghost entities despawned. Game pauses with 'Host Lost' overlay. No host migration in v1.0.	Full session loss. Players keep their local save state.
Client crash during zone handoff	Handoff ACK not received within 2 seconds	Host aborts handoff. Zone authority reverts to host. NPCs unfrozen.	NPCs may snap to host's last known state. Brief visual glitch.
Desync detected (state hash mismatch)	Periodic audit (every 30 sec) compares entity state hashes	Host sends full state correction for desynced entities. Clients apply and snap-correct.	Brief visual corrections. Game state is restored to host's authoritative version.
Memory read fails (invalid game pointer)	Access violation / exception in OnReadTransform	Entity is flagged as invalid and despawned on next tick. Structured exception handler prevents crash.	Entity disappears. Can be respawned if the game entity is recreated.

12. Performance Analysis

12.1 CPU Cost Per Frame

Operation	Per Entity	8 Players + 40 NPCs	Notes
TickLocal (read + dirty check)	~0.005 ms	~0.05 ms (10 local entities)	Simple float comparisons
TickGhost (interpolation + write)	~0.008 ms	~0.30 ms (38 ghosts)	Sort + lerp + memory write
InterpBuffer::Sample()	~0.003 ms	Included above	Sort of 8 elements + lerp
OnSendStateUpdate (serialize)	~0.010 ms	~0.10 ms (10 local, when dirty)	memcpy into packet buffer
Zone recalc (host, 1/sec)	~0.05 ms	~0.05 ms	Only runs once per second
Total per frame		~0.5 ms	Well within 16ms frame budget

12.2 Memory Usage

Component	Size Per Entity	Total (200 entities)	Notes
NetworkedEntity struct	~530 bytes	~106 KB	Includes all sub-structures
InterpBuffer (8 snapshots)	~320 bytes	~64 KB (ghosts only)	Part of NetworkedEntity
Registry overhead (hash map)	~48 bytes/entry	~9.6 KB	Bucket + pointer overhead
Zone authority map	~32 bytes/zone	~8 KB (256 active zones)	ZoneCoord + uint32
Total		~188 KB	Negligible

12.3 Network Bandwidth

Scenario	Entities Sending	Upstream Per Client	Host Total Upstream
2 players, no combat	2 (players only)	~5 kbps	~5 kbps
4 players, light combat	4 players + 20 NPCs	~15 kbps	~45 kbps
8 players, heavy combat	8 players + 50 NPCs	~35 kbps	~250 kbps
Worst case (all dirty)	8 players + 100 NPCs	~65 kbps	~500 kbps

All scenarios are well within typical home internet upload bandwidth (5-20 Mbps).

13. Memory Layout and Structure Sizes

All network packet structures use `#pragma pack(push, 1)` to eliminate padding. This ensures consistent wire format across compilers. Structure sizes:

Structure	Packed Size	Fields	Alignment
PacketHeader	8 bytes	type(1) + flags(1) + tick(2) + netId(4)	1-byte packed
PosUpdatePayload	44 bytes	pos(12) + rot(16) + vel(12) + anim(2) + blend(4) + speed(4) - 6 padding removed	1-byte packed
EntitySpawnPayload	105 bytes	netId(4) + type(1) + owner(4) + pos(12) + rot(16) + health(4) + name(32) + model(32)	1-byte packed
DamageEventPayload	13 bytes	attacker(4) + target(4) + damage(4) + limbId(1)	1-byte packed
CombatStatePayload	8 bytes	target(4) + attack(1) + block(1) + animId(2)	1-byte packed
HealthUpdatePayload	4 bytes	health(4)	1-byte packed
LimbDamagePayload	5 bytes	limbId(1) + newHp(4)	1-byte packed

Internal structures (not sent over network) use natural alignment:

Structure	Approx. Size	Notes
Vec3	12 bytes	3 floats, naturally aligned.
Quat	16 bytes	4 floats.
EntitySnapshot	~40 bytes	double(8) + uint16(2) + Vec3(12) + Quat(16) + Vec3(12) + uint16(2) + float(4) + float(4) - with padding.
InterpBuffer	~328 bytes	8 x EntitySnapshot + size_t writeldx + size_t count.
JitterEstimator	~40 bytes	5 doubles.
LimbHealth	28 bytes	7 floats.
CombatInfo	~16 bytes	uint32 + 2 uint8 + uint16 + float, with padding.
NetworkedEntity	~530 bytes	All fields combined with string and sub-structures.

14. API Reference

EntityManager::Get()

Returns: static EntityManager&

Description: Returns the singleton instance.

Notes: Thread-safe (static local).

Initialize(isHost, localPeerId)

Returns: void

Description: Initializes the engine. Clears registry, sets up ID counters, records peer ID.

Notes: Call once at mod startup. Not thread-safe — call before any other threads access the manager.

Shutdown()

Returns: void

Description: Despawns all ghosts, clears registry.

Notes: Call on mod unload. Not thread-safe.

SetCallbacks(cb)

Returns: void

Description: Assigns the callback struct.

Notes: Call immediately after Initialize.

AllocatePlayerNetId()

Returns: uint32_t

Description: Returns next available player NetworkID. Host only.

Notes: Returns NETID_INVALID if range exhausted.

AllocateNPCNetId()

Returns: uint32_t

Description: Returns next available NPC NetworkID. Host only.

Notes: Returns NETID_INVALID if range exhausted.

AllocateBuildingNetId()

Returns: uint32_t

Description: Returns next available building NetworkID. Host only.

Notes: Returns NETID_INVALID if range exhausted.

RegisterLocalPlayer(ptr, name, assignedId)

Returns: uint32_t

Description: Registers the local player entity. Reads initial state, broadcasts spawn.

Notes: Call once from player controller hook.

RegisterLocalNPC(ptr, name, squadId, assignedId)

Returns: uint32_t

Description: Registers a local NPC entity.

Notes: Call for each squad member or owned NPC.

SpawnRemoteEntity(netId, type, ownerPeerId, pos, rot, name, modelId)

Returns: bool

Description: Creates a ghost entity for a remote player/NPC. Calls OnSpawnGhost.

Notes: Called from network receive when ENTITY_SPAWN arrives.

DespawnEntity(netId)

Returns: void

Description: Removes an entity from the registry. Despawns ghost if applicable.

Notes: Called from network receive or authority system.

DespawnPeerEntities(peerId)

Returns: void

Description: Bulk despawn all ghosts owned by a disconnected peer.

Notes: Called from ENet disconnect handler.

OnPositionUpdate(...)

Returns: void

Description: Pushes a position snapshot into a ghost's interpolation buffer.

Notes: Called from packet receive handler for POS_UPDATE.

OnHealthUpdate(netId, health)

Returns: void

Description: Updates a ghost's health value.

Notes: Called for HEALTH_UPDATE packets.

OnLimbDamage(netId, limbId, newHp)

Returns: void

Description: Updates a specific limb's HP on a ghost.

Notes: Called for LIMB_DAMAGE packets.

OnCombatStateUpdate(...)

Returns: void

Description: Updates combat info on a ghost entity.

Notes: Called for COMBAT_STATE packets.

Tick(deltaTime)

Returns: void

Description: Main frame update. Ticks all entities, runs interpolation, sends dirty state.

Notes: Call once per frame from game thread.

GetEntity(netId)

Returns: NetworkedEntity*

Description: Direct lookup by NetworkID.

Notes: Returns nullptr if not found.

GetLocalPlayer()

Returns: NetworkedEntity*

Description: Returns the local player entity.

Notes: Shortcut for GetEntity(localPlayerNetId).

GetWorldSnapshot()

Returns: vector

Description: Serializes all active entities for a joining client.

Notes: Host calls this during client handshake.

SetServerTime(time) / GetServerTime()

Returns: void / double

Description: Sets or gets the synchronized server time.

Notes: Host sends time; clients sync to it.

15. Implementation Checklist

Build and test these components in order:

#	Component	Est. Effort	Test Strategy
1	Vec3, Quat, ZoneCoord math types	2 hours	Unit tests for lerp, slerp, distance, zone calculation.
2	EntitySnapshot and InterpBuffer	4 hours	Unit tests for push, sample, interpolation, extrapolation, edge cases (empty, single, full).
3	JitterEstimator	2 hours	Feed simulated packet arrivals. Verify adaptive delay matches expected ranges.
4	DirtyFlags bitfield operations	1 hour	Unit tests for OR, AND, HasFlag.
5	NetworkedEntity struct + state machine	2 hours	Verify state transitions. Test all helper methods (IsPlayer, IsLocal, etc).
6	EntityManager singleton + Initialize/Shutdown	2 hours	Verify clean init, shutdown, re-init cycle.
7	ID allocation (AllocatePlayerNetId, etc.)	1 hour	Verify range boundaries. Test exhaustion.
8	RegisterLocalPlayer + RegisterLocalNPC	4 hours	Mock OnReadTransform. Verify registry insertion, callback firing.
9	SpawnRemoteEntity (ghost system)	6 hours	Mock OnSpawnGhost. Verify ghost creation, interp buffer init, idempotency.
10	DespawnEntity + DespawnPeerEntities	3 hours	Verify ghost cleanup, broadcast, bulk despawn.
11	OnPositionUpdate + tick ordering	4 hours	Feed mock packets. Verify interp buffer population. Test out-of-order rejection.
12	TickGhost (interpolation + snap correction)	6 hours	Feed known snapshots. Verify interpolated output matches expected values.
13	TickLocal (dirty detection + send)	4 hours	Mock OnReadTransform with changing data. Verify dirty flags and send timing.
14	Zone authority recalculation	4 hours	Place mock players. Verify zone assignments, hysteresis, handoff callbacks.
15	Thread safety audit	4 hours	Run concurrent read/write stress test. Verify no crashes or data corruption.
16	Integration with real hooks + ENet	8 hours	End-to-end test: two game instances, one hosts, one joins.
17	GetWorldSnapshot + client join flow	4 hours	Host creates entities, snapshot sent to mock client, verify reconstruction.
18	Error handling edge cases	4 hours	Test every error scenario from Section 11.

Total estimated effort: ~60 hours (approximately 2 weeks for one developer).

END OF DOCUMENT

Kenshi Online — Entity Management Engine — Version 1.0